

because they still “work.” Yet, while those habits persist, Python has quietly evolved, introducing subtle but powerful conveniences that often go unnoticed.

The following table describes the usage of Python operators/features in the old and the new patterns, and when to use them.

Operator/Feature	Old pattern	New pattern	Usage scenario
Walrus Operator (:=)	<pre>line = f.readline() while line: process(line) line = f.readline()</pre>	<pre>while (line := f.readline()): process(line)</pre>	Reading files, sockets, or buffers; Reduces duplication and prevents subtle bugs during refactoring
pathlib	<pre>import os p = os.path. join("data", "logs", "2024", "today.txt")</pre>	<pre>from pathlib import Path p = Path("data") / "logs" / "2024" / "today.txt"</pre>	Any script that manipulates files or folders; Cross-platform projects
Dictionary Merge ()	<pre>merged = base.copy() merged. update(overrides)</pre>	<pre>merged = base overrides</pre>	Config layering; Merging dictionaries without mutating originals
f-String Debugging	<pre>print("result =", result)</pre>	<pre>print(f"{result=}")</pre>	Removes small but frequent debug- ging annoyances; Logging temporary values; Printing multiple variables
Dataclasses	<pre>class Product: def __init__(self, name, price): self.name = name self.price = price</pre>	<pre>from dataclasses import dataclass @dataclass class Product: name: str price: float</pre>	Data containers (ETL rows, JSON models, config objects); Cases where you want readable print/ debug outputs
perf_counter()	<pre>import time start = time.time()</pre>	<pre>from time import perf_counter start = perf_counter()</pre>	Comparing two algorithms, measuring database calls, or tuning ML preproc- essing; Performance diagnostics
Structural pattern matching	<pre>if event["type"] == "login": ... elif event["type"] == "logout": ...</pre>	<pre>match event: case {"type": "login", "user": u}: handle_login(u) case {"type": "logout", "user": u}: handle_logout(u)</pre>	Handling multiple event types; Parsing structured data (JSON, logs); Readability is collapsing under if/elif chains
DuckDB	<pre>import pandas as pd df = pd.read_ csv("large.csv")</pre>	<pre>import duckdb df = duckdb.query("SELECT * FROM 'large.csv'").df()</pre>	Loading large CSVs or Parquet files; Running analytics without putting strain on memory; Exploratory work where SQL is more natural than Pandas
with statements	<pre>f = open("data.txt") data = f.read() f.close()</pre>	<pre>with open("data.txt") as f: data = f.read()</pre>	File handles, DB connections, sock- ets, locks
enumerate()	<pre>i = 0 for item in items: print(i, item) i += 1</pre>	<pre>for i, item in enumerate(items): print(i, item)</pre>	Listing items, debugging loops, gen- erating numbered outputs